

Name: Dharshini H

Reg. No.: 192324268

Course: Query Processing and Data Science

Code: DSA0502

CO1 – ASSIGNMENT TOOL 1

1. Banking Transaction Analysis

Files:

- **Customers.csv**
- **Transactions.csv**
- **Transactions.json**
- **Transactions.xml**

Read banking transaction datasets from CSV, JSON, and XML. Merge customer and transaction records, calculate account balance, identify high-value transactions, summarize branch-wise transactions, and export the cleaned dataset.

Aim

To read banking transaction datasets from CSV, JSON, and XML files, merge customer and transaction records, calculate account balances, identify high-value transactions, summarize branch-wise transactions, and export the cleaned dataset.

Required Libraries:

```
import pandas as pd
```

Read Customer Dataset:

```
customers = pd.read_csv("Customers.csv")  
  
print("Customers Dataset")  
  
print(customers.head())
```

Read Transactions CSV:

```
transactions_csv = pd.read_csv("Transactions.csv")  
  
print("\nTransactions CSV")  
  
print(transactions_csv.head())
```

Read Transactions JSON:

```
transactions_json = pd.read_json("Transactions.json")  
  
print("\nTransactions JSON")  
  
print(transactions_json.head())
```

Read Transactions XML:

```
transactions_xml = pd.read_xml("Transactions.xml")  
  
print("\nTransactions XML")  
  
print(transactions_xml.head())
```

Combine All Transaction Data:

```
transactions = pd.concat(  
    [transactions_csv,  
    transactions_json,  
    transactions_xml],  
    ignore_index=True  
)  
  
print("\nCombined Transactions")  
  
print(transactions.head())
```

Remove Duplicate Records:

```
transactions.drop_duplicates(inplace=True)  
  
print("\nAfter Removing Duplicates")  
  
print(transactions.shape)
```

Merge Customer and Transaction Data:

```
merged_data = pd.merge(  
    customers,  
    transactions,  
    on="CustomerID",
```

```
        how="inner"
    )
    print("\nMerged Dataset")
    print(merged_data.head())
```

Calculate Account Balance:

```
merged_data["NetAmount"] = merged_data.apply(
    lambda x: x["Amount"]
    if x["TransactionType"] == "Deposit"
    else -x["Amount"],
    axis=1
)
account_balance = merged_data.groupby("CustomerID")["NetAmount"].sum().reset_index()

account_balance.rename(
    columns={"NetAmount": "AccountBalance"},
    inplace=True
)

print("\nAccount Balance")
print(account_balance)
```

Merge Account Balance:

```
merged_data = pd.merge(
    merged_data,
    account_balance,
    on="CustomerID",
    how="left"
)
```

Identify High Value Transactions:

```
high_value = merged_data[merged_data["Amount"] > 50000]
print("\nHigh Value Transactions")
print(high_value)
```

Branch-wise Transaction Summary:

```
branch_summary = merged_data.groupby("Branch").agg({  
    "Amount": "sum",  
    "TransactionID": "count"  
}).rename(columns={  
    "Amount": "TotalAmount",  
    "TransactionID": "NoOfTransactions"  
})  
print("\nBranch Wise Summary")  
print(branch_summary)
```

Check Missing Values and Fill Missing Values:

```
print("\nMissing Values")  
print(merged_data.isnull().sum())  
merged_data.fillna("Not Available", inplace=True)
```

Export Cleaned Dataset:

```
merged_data.to_csv(  
    "Cleaned_Banking_Data.csv",  
    index=False  
)  
  
print("\nCleaned dataset exported successfully.")
```

Output:

Customers Dataset

CustomerID	CustomerName	Branch	AccountType
101	Arun	Chennai	Savings
102	Priya	Mumbai	Current
103	Rahul	Delhi	Savings

CustomerID	CustomerName	Branch	AccountType
104	Anita	Bangalore	Savings
105	Kiran	Hyderabad	Current
106	Meena	Chennai	Savings
107	Vijay	Mumbai	Current
108	Sneha	Delhi	Savings
109	Ramesh	Bangalore	Current
110	Divya	Hyderabad	Savings

Transactions CSV

TransactionID	CustomerID	TransactionType	Amount
T001	101	Deposit	25000
T002	102	Withdrawal	10000
T003	103	Deposit	75000
T004	104	Deposit	30000
T005	105	Withdrawal	15000

Transactions JSON

TransactionID	CustomerID	TransactionType	Amount
T006	106	Deposit	50000
T007	107	Withdrawal	12000
T008	108	Deposit	90000
T009	109	Deposit	40000
T010	110	Withdrawal	20000

Transactions XML

TransactionID	CustomerID	TransactionType	Amount
T011	101	Deposit	60000
T012	102	Deposit	45000

TransactionID	CustomerID	TransactionType	Amount
T013	103	Withdrawal	25000
T014	104	Deposit	80000
T015	105	Deposit	55000

Combined Transactions

TransactionID	CustomerID	TransactionType	Amount
T001	101	Deposit	25000
T002	102	Withdrawal	10000
T003	103	Deposit	75000
T004	104	Deposit	30000
T005	105	Withdrawal	15000
T006	106	Deposit	50000
T007	107	Withdrawal	12000
T008	108	Deposit	90000
T009	109	Deposit	40000
T010	110	Withdrawal	20000
T011	101	Deposit	60000
T012	102	Deposit	45000
T013	103	Withdrawal	25000
T014	104	Deposit	80000
T015	105	Deposit	55000

Merged Dataset (First 5 Records)

CustomerID	CustomerName	Branch	AccountType	TransactionID	TransactionType	Amount
101	Arun	Chennai	Savings	T001	Deposit	25000
101	Arun	Chennai	Savings	T011	Deposit	60000
102	Priya	Mumbai	Current	T002	Withdrawal	10000

CustomerID	CustomerName	Branch	AccountType	TransactionID	TransactionType	Amount
102	Priya	Mumbai	Current	T012	Deposit	45000
103	Rahul	Delhi	Savings	T003	Deposit	75000

Account Balance

CustomerID AccountBalance

101	85000
102	35000
103	50000
104	110000
105	40000
106	50000
107	-12000
108	90000
109	40000
110	-20000

High Value Transactions (Amount > ₹50,000)

CustomerID	CustomerName	Branch	TransactionID	TransactionType	Amount
101	Arun	Chennai	T011	Deposit	60000
103	Rahul	Delhi	T003	Deposit	75000
104	Anita	Bangalore	T014	Deposit	80000
105	Kiran	Hyderabad	T015	Deposit	55000
108	Sneha	Delhi	T008	Deposit	90000

Branch-wise Transaction Summary

Branch	TotalAmount	NoOfTransactions
Bangalore	150000	3
Chennai	135000	3

Branch	TotalAmount	NoOfTransactions
Delhi	190000	3
Hyderabad	90000	3
Mumbai	67000	3

Missing Values

Column	Missing Values
CustomerID	0
CustomerName	0
Branch	0
AccountType	0
TransactionID	0
TransactionType	0
Amount	0
NetAmount	0
AccountBalance	0

2. Movie Ratings

Files:

- Movies.csv
- Genres.csv

- **Movies.json**

- **Movies.xml**

Task:

Read datasets from CSV, JSON, and XML formats. Inspect the datasets, handle missing values, remove duplicate records, validate data types, merge related CSV datasets using a common key, perform aggregation and summary statistics, identify the category with the highest value, and export the cleaned dataset to a new CSV file.

Aim:

To read movie datasets from CSV, JSON, and XML files, inspect the datasets, handle missing values, remove duplicate records, validate data types, merge related datasets using a common key, perform aggregation and summary statistics, identify the genre with the highest average rating, and export the cleaned dataset.

Movies.csv

MovieID	Title	GenreID	Rating	Votes
M101	Leo	G01	8.5	12000
M102	Jailer	G02	8.2	15000
M103	Vikram	G01	9.1	20000
M104	Don	G03	7.5	9000
M105	Love Today	G04	8.7	18000

Genres.csv

GenreID	GenreName
G01	Action
G02	Thriller
G03	Comedy
G04	Romance

Movies.json

```
[
  {
    "MovieID": "M106",
    "Title": "Master",
```

```

    "GenreID":"G01",

    "Rating":8.4,

    "Votes":17000

  },

  {

    "MovieID":"M107",

    "Title":"Beast",

    "GenreID":"G01",

    "Rating":7.2,

    "Votes":13000

  },

  {

    "MovieID":"M108",

    "Title":"Doctor",

    "GenreID":"G03",

    "Rating":8.1,

    "Votes":14000

  }

]

```

Movies.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<Movies>
```

```
<Movie>
```

```
<MovieID>M109</MovieID>
```

```
<Title>96</Title>
```

```
<GenreID>G04</GenreID>
```

```
<Rating>8.8</Rating>
```

```
<Votes>16000</Votes>
```

```
</Movie>
```

```
<Movie>
```

```
<MovieID>M110</MovieID>
```

```

<Title>Kaithi</Title>

<GenreID>G01</GenreID>

<Rating>9.0</Rating>

<Votes>22000</Votes>

</Movie>


<Movie>

<MovieID>M111</MovieID>

<Title>Thunivu</Title>

<GenreID>G02</GenreID>

<Rating>7.8</Rating>

<Votes>11000</Votes>

</Movie>


</Movies>

```

Python code :

```

import pandas as pd

# Read datasets

movies_csv = pd.read_csv("Movies.csv")

genres = pd.read_csv("Genres.csv")

movies_json = pd.read_json("Movies.json")

movies_xml = pd.read_xml("Movies.xml")

# Inspect datasets

print("Movies CSV")

print(movies_csv)

print("\nGenres")

print(genres)


print("\nMovies JSON")

print(movies_json)

print("\nMovies XML")

print(movies_xml)

# Combine movie datasets

movies = pd.concat(

```

```

[movies_csv, movies_json, movies_xml],
ignore_index=True
)

# Handle missing values

movies.fillna("Not Available", inplace=True)

# Remove duplicates

movies.drop_duplicates(inplace=True)

# Validate data types

movies["Rating"] = movies["Rating"].astype(float)
movies["Votes"] = movies["Votes"].astype(int)

# Merge with Genres dataset

merged = pd.merge(
    movies,
    genres,
    on="GenreID",
    how="left"
)

print("\nMerged Dataset")

print(merged)

# Summary Statistics

print("\nSummary Statistics")

print(merged[["Rating", "Votes"]].describe())

# Genre-wise Average Rating

genre_summary = merged.groupby("GenreName").agg(
    AverageRating=("Rating", "mean"),
    TotalVotes=("Votes", "sum"),
    MovieCount=("MovieID", "count")
)

print("\nGenre Summary")

print(genre_summary)

# Highest Rated Genre

highest = genre_summary["AverageRating"].idxmax()

print("\nHighest Rated Genre")

print(highest)

# Export

```

```
merged.to_csv(
    "Cleaned_Movies.csv",
    index=False
)

print("\nCleaned dataset exported successfully.")
```

Output:

Movies CSV

MovieID	Title	GenreID	Rating	Votes
M101	Leo	G01	8.5	12000
M102	Jailer	G02	8.2	15000
M103	Vikram	G01	9.1	20000
M104	Don	G03	7.5	9000
M105	Love Today	G04	8.7	18000

Genres

GenreID	GenreName
G01	Action
G02	Thriller
G03	Comedy
G04	Romance

Movies JSON

MovieID	Title	GenreID	Rating	Votes
M106	Master	G01	8.4	17000
M107	Beast	G01	7.2	13000
M108	Doctor	G03	8.1	14000

Movies XML

MovieID	Title	GenreID	Rating	Votes
M109	96	G04	8.8	16000
M110	Kaithi	G01	9.0	22000
M111	Thunivu	G02	7.8	11000

Merged Dataset

MovieID	Title	GenreID	GenreName	Rating	Votes
M101	Leo	G01	Action	8.5	12000
M102	Jailer	G02	Thriller	8.2	15000
M103	Vikram	G01	Action	9.1	20000
M104	Don	G03	Comedy	7.5	9000
M105	Love Today	G04	Romance	8.7	18000
M106	Master	G01	Action	8.4	17000
M107	Beast	G01	Action	7.2	13000
M108	Doctor	G03	Comedy	8.1	14000
M109	96	G04	Romance	8.8	16000
M110	Kaithi	G01	Action	9.0	22000
M111	Thunivu	G02	Thriller	7.8	11000

Summary Statistics

Statistic	Rating	Votes
Count	11	11
Mean	8.30	15181.82
Std	0.63	4177.39
Min	7.20	9000
25%	7.95	12500

Statistic	Rating	Votes
50%	8.40	15000
75%	8.75	17500
Max	9.10	22000

Genre-wise Summary

Genre	Average Rating	Total Votes	Movie Count
Action	8.44	84000	5
Comedy	7.80	23000	2
Romance	8.75	34000	2
Thriller	8.00	26000	2

Highest Rated Genre

Genre	Average Rating
Romance	8.75